

TP2 — Analyse d'un championnat de football avec Pandas

Oct 12, 2026 · @Elias

Python Avancé · Licence 3 IA · ISM École d'ingénieurs · Enseignant : Elias BADJI

Contexte et consignes

Vous êtes analyste pour la Ligue Pro, un championnat de football fictif de 20 clubs. La direction vous transmet l'historique des trois dernières saisons (2023-2024, 2024-2025 et 2025-2026) : 38 journées par saison, 380 matchs par saison, chaque but enregistré avec sa minute, son buteur et son passeur. Les fichiers ont été saisis à la main et contiennent des erreurs. Vous devez les nettoyer, reconstruire les classements, puis produire les indicateurs du rapport de fin de saison. Les clubs, les joueurs et les résultats sont entièrement inventés.

Objectifs. Charger et diagnostiquer quatre fichiers de plus de 1000 lignes, nettoyer des données mal saisies, transformer une table de matchs en classement, croiser des tables avec `merge`, agréger avec `groupby` sur plusieurs clés et exporter des résultats.

Organisation. Durée 2h, en binôme. Un seul notebook par binôme, nommé `TP2_NOM1_NOM2.ipynb`, avec les fichiers dans un sous-dossier `data/`.

Règles.

- Pandas et NumPy uniquement : aucune boucle `for` sur les lignes d'un DataFrame.
- Chaque choix de nettoyage est justifié en une phrase dans une cellule Markdown.
- Chaque résultat est suivi d'une phrase d'interprétation.
- Le notebook doit s'exécuter de bout en bout après « Redémarrer et tout exécuter ».

Rendu. En fin de séance : le lien du notebook, et le fichier Excel produit à la partie 5.

Barème (sur 20).

Partie	Thème	Points
1	Chargement et diagnostic	3
2	Nettoyage	4
3	Classements des trois saisons	5
4	Analyses	6
5	Export	1
—	Qualité des commentaires	1
Bonus	Course au titre journée par journée	+2

Les données

Quatre fichiers CSV, à récupérer dans le dossier `data/` du dépôt du cours (github.com/Techbadji/pythonavance). Ils sont reliés par les clés `equipe_id`, `joueur_id`, `match_id` et `saison`. Seul `equipes.csv` est fiable ; les trois autres contiennent des erreurs de saisie, à vous de les trouver.

Fichier	Lignes	Une ligne par…
equipes.csv	20	club
matchs.csv	1 145	match
joueurs.csv	1 688	joueur et par saison
buts.csv	2 660	but marqué

equipes.csv

Colonne	Description
equipe_id	Identifiant du club (E01 à E20)
nom, ville, region	Nom du club et localisation
annee_creation	Année de fondation
stade, capacite_stade	Stade et nombre de places

Colonne	Description
budget_millions_fcfa	Budget annuel en millions de FCFA

matchs.csv

Colonne	Description
match_id, saison, journee	Identifiant du match, saison et journée (1 à 38)
date, heure	Date et heure du coup d'envoi
equipe_dom_id, equipe_ext_id	Club qui reçoit et club qui se déplace
buts_dom, buts_ext	Score final
tirs_dom, tirs_ext	Nombre de tirs de chaque équipe
possession_dom_pct	Possession du club qui reçoit, en %
spectateurs	Affluence au stade

joueurs.csv : un joueur apparaît une fois par saison jouée dans la Ligue Pro, avec son club et ses statistiques de cette saison.

Colonne	Description
saison	Saison concernée
joueur_id, prenom, nom	Identifiant stable d'une saison à l'autre, et identité
equipe_id	Club du joueur cette saison-là
poste	Gardien, Défenseur, Milieu ou Attaquant
date_naissance, nationalite	État civil
salaire_mensuel_fcfa	Salaire mensuel en FCFA
matchs_joues, minutes_jouees	Temps de jeu sur la saison
buts, passes_decisives	Statistiques offensives de la saison
cartons_jaunes, cartons_rouges	Sanctions reçues

buts.csv

Colonne	Description
but_id, match_id, saison	Identifiant du but, du match et saison
minute	Minute du but, en texte : « 37 », ou « 45+2 » et « 90+3 » pour le temps additionnel
equipe_id	Club qui a marqué
buteur_id, passeur_id	Joueurs concernés ; passeur vide s'il n'y a pas de passe décisive
type_but	« jeu » ou « penalty »

Aperçu des premières lignes de `buts.csv` :

```
but_id,match_id,saison,minute,equipe_id,buteur_id,passeur_id,type_but
B00001,M0001,2023-2024,74,E01,J0023,J0021,jeu
B00002,M0001,2023-2024,15,E20,J0541,,penalty
```

Partie 1 — Chargement et diagnostic (3 pts, 15 min)

1.1 Chargez les quatre fichiers dans `equipes`, `matchs`, `joueurs` et `buts`. Lisez la colonne `date` de `matchs` comme une date.

1.2 Pour chaque DataFrame, affichez sa forme et ses types avec `info()`.

1.3 Dans `joueurs`, la colonne `salaire_mensuel_fcfa` devrait être numérique. Quel est son type ? Affichez quelques valeurs qui empêchent la conversion et expliquez le problème.

1.4 Comptez les valeurs manquantes par colonne dans les quatre tables. Dans `buts`, les `passeur_id` manquants sont-ils une erreur ?

1.5 Comptez les lignes en double dans chaque fichier, puis les clés en double : `match_id`, `but_id`, et le couple (`saison`, `joueur_id`).

1.6 Affichez les valeurs distinctes de `poste` avec leurs effectifs. Combien de variantes trouvez-vous pour quatre postes ?

1.7 Combien de buts ont une minute du type « 90+3 » ? Indice : `.str.contains(r"\+")`.

1.8 Rédigez en Markdown la liste des problèmes trouvés, un par ligne.

Partie 2 — Nettoyage (4 pts, 20 min)

Travaillez sur des copies, pour pouvoir comparer avant et après nettoyage à la question 4.8.

2.1 Supprimez les doublons des trois tables concernées. Vérifiez ensuite que chaque `match_id`, chaque `but_id` et chaque couple (`saison`, `joueur_id`) est unique.

2.2 Harmonisez `poste` pour n'avoir que quatre valeurs : Gardien, Défenseur, Milieu, Attaquant. Indice : `.str.strip()`, `.str.capitalize()` puis `.replace()`.

2.3 Convertissez `salaire_mensuel_fcfa` en nombre : retirez les espaces avec `.str.replace(" ", "")`, puis utilisez `pd.to_numeric`.

2.4 Remplacez chaque salaire manquant par le salaire **médian des joueurs du même poste et de la même saison**. Indice : `groupby(["saison", "poste"])["salaire_mensuel_fcfa"].transform("median")`. Pourquoi ces deux critères plutôt que la médiane de tous les joueurs ?

2.5 Convertissez `date_naissance` en date. Faut-il remplacer les dates manquantes ? Justifiez votre choix.

2.6 Dans `buts`, créez deux colonnes numériques à partir de `minute` : `minute_num` (la minute réglementaire : 45 pour « 45+2 ») et `temps_add` (les minutes additionnelles : 2 pour « 45+2 », 0 sinon). Indice : `.str.split("+", expand=True)`.

2.7 Proposez et justifiez un traitement pour les spectateurs manquants de `matchs`.

2.8 Contrôle final : vous devez obtenir 1 140 matchs, 1 680 lignes joueur-saison et 2 654 buts.

Partie 3 — Classements des trois saisons (5 pts, 30 min)

Un match concerne deux équipes, mais un classement compte une ligne par équipe. Il faut donc transformer chaque match en **deux lignes**, une par équipe, vue de son côté :

match_id	equipe_dom_id	equipe_ext_id	buts_dom	buts_ext
M0001	E01	E20	1	2

devient

match_id	equipe_id	buts_pour	buts_contre	points
M0001	E01	1	2	0

match_id	equipe_id	buts_pour	buts_contre	points
M0001	E20	2	1	3

3.1 Construisez un DataFrame `dom` avec, pour chaque match, l'équipe qui reçoit : colonnes `saison`, `journee`, `equipe_id`, `buts_pour`, `buts_contre`. Utilisez `rename`.

3.2 Construisez de même `ext` pour l'équipe qui se déplace : ses buts pour sont `buts_ext`, ses buts contre `buts_dom`.

3.3 Empilez-les avec `pd.concat` dans `resultats`. Combien de lignes doit-il contenir ? Vérifiez.

3.4 Ajoutez les colonnes `points` (3, 1 ou 0), `victoire`, `nul` et `defaite` (1 ou 0). Indice : `np.select`, ou des comparaisons converties avec `.astype(int)`.

3.5 Avec une agrégation nommée sur les clés `["saison", "equipe_id"]`, calculez pour chaque club et chaque saison : J, G, N, P, BP, BC et Pts. Ajoutez Diff = BP − BC.

3.6 Ajoutez le nom des clubs avec `merge`, puis triez selon le règlement : saison croissante, puis points, différence de buts et buts marqués décroissants. Ajoutez une colonne `rang` de 1 à 20 dans chaque saison. Indice : `groupby("saison").cumcount() + 1`.

3.7 Contrôles de cohérence avec `assert` :

- chaque club a joué 38 matchs dans chaque saison ;
- dans chaque saison, la somme des buts pour est égale à la somme des buts contre ;
- dans chaque saison, le total des victoires est égal au total des défaites.

3.8 Affichez le podium de chaque saison. Un club est-il sur le podium les trois saisons ?

3.9 Affichez le classement complet de 2025-2026. Deux clubs y terminent à égalité de points **et** de différence de buts : lesquels, et lequel est devant ?

Partie 4 — Analyses pour le rapport (6 pts, 45 min)

Répondez à chaque question par un tableau ou une valeur, suivi d'une phrase d'interprétation. Sauf mention contraire, travaillez sur les trois saisons.

4.1 Meilleurs buteurs. Affichez les 5 meilleurs buteurs de la saison 2025-2026 avec leur prénom, leur nom, le **nom** de leur club, leurs buts et leurs passes décisives.

4.2 Avantage du terrain. Pour chaque saison, quelle part des matchs se termine par une victoire à domicile, un nul, une victoire à l'extérieur ? Indice : `pd.crosstab(..., normalize="index")`. L'avantage du terrain est-il stable d'une saison à l'autre ?

4.3 Affluence. Pour la saison 2025-2026, calculez l'affluence moyenne de chaque club à domicile et le taux de remplissage de son stade, en %. Quels sont les trois clubs qui remplissent le mieux leur stade ?

4.4 Quand marque-t-on ? Avec `pd.cut` sur `minute_num`, répartissez les buts en tranches de 15 minutes (1-15, 16-30, …, 76-90) et calculez la part de chaque tranche. Quelle part des buts est marquée dans le temps additionnel ? Commentez.

4.5 Penalties et passes. Quelle part des buts vient d'un penalty ? Quelle part des buts n'a pas de passeur ? Trouvez ensuite les trois duos passeur → buteur les plus prolifiques, avec les **noms** des deux joueurs. Indice : `groupby(["passeur_id", "buteur_id"]).size()`, puis deux `merge` avec une table d'identité des joueurs.

4.6 Argent et résultats. Pour 2025-2026, calculez la masse salariale mensuelle de chaque club, joignez-la au classement et calculez la corrélation entre masse salariale et points. Que peut-on conclure, et que ne peut-on pas conclure ?

4.7 Carrières. Combien de joueurs différents compte la Ligue sur trois saisons ? Combien ont joué les trois saisons ? Combien ont porté le maillot de plusieurs clubs ? Indice : `groupby("joueur_id").agg(...)` avec `nunique`.

4.8 Contrôles croisés. Vérifiez avec les tables nettoyées que :

- pour chaque match et chaque équipe, le nombre de lignes de `buts` est égal au score de `matchs` ;
- pour chaque joueur et chaque saison, la colonne `buts` de `joueurs` est égale au nombre de buts qu'il a marqués dans `buts`.

Refaites le premier contrôle avec le fichier `buts.csv` brut, avant nettoyage : combien de scores ne correspondent plus, et pourquoi ?

Partie 5 — Export (1 pt, 5 min)

5.1 Exportez dans un seul fichier `rapport_ligue.xlsx` trois feuilles : `Classements` (les trois saisons), `Buteurs_2025-2026` et `Affluence_2025-2026`. Indice :

```
with pd.ExcelWriter("rapport_ligue.xlsx") as writer:
    classements.to_excel(writer, sheet_name="Classements", index=False)
    buteurs.to_excel(writer, sheet_name="Buteurs_2025-2026", index=False)
```

Bonus — La course au titre 2025-2026 (+2 pts)

B.1 À partir de `resultats`, pour la saison 2025-2026, calculez les points cumulés de chaque club après chaque journée. Indice : triez par journée, puis `groupby("equipe_id")["points"].cumsum()`.

B.2 Avec `pivot_table`, construisez un tableau avec une ligne par journée et une colonne par club. Quel club était en tête après chaque journée ? Combien de fois la première place a-t-elle changé de mains ?

B.3 Gardez ce tableau : il servira à la séance 3 pour tracer la course au titre.